

LINUX: Servidor DHCP (Debian 12)

Servicios en Red e Internet · Versión 2 (septiembre de 2026)

Nota: todas las operaciones se hacen desde un terminal como administrador, preferiblemente anteponiendo **sudo** a cada orden.

Los nombres de interfaces y direcciones IP de los ejemplos pueden no coincidir con los de vuestro esquema de red. Adaptadlos siempre.

1. Configuración de las interfaces de red en Debian 12

Antes de montar cualquier servicio de red, el servidor tiene que tener su red bien configurada. En este apartado vemos cómo hacerlo **desde el terminal**, qué ficheros intervienen y cómo comprobar el resultado.

1.1. Identificar las interfaces

```
ip -br link          # interfaces y su estado (UP/DOWN) y MAC
ip -br address       # IP asignadas a cada interfaz
ip route             # tabla de rutas (la "default" es la puerta de enlace)
```

Ejemplo de salida:

lo	UNKNOWN	127.0.0.1/8 ::1/128
ens18	UP	10.3.129.57/24 fe80::be24:11ff:fe4e:66a1/64
ens19	UP	192.168.29.1/24 fe80::be24:11ff:fe9b:1c2d/64

Los nombres de las interfaces dependen de dónde corre el sistema:

- **Máquina virtual o equipo físico:** Debian usa **nombres predecibles** según la posición de la tarjeta. Por ejemplo, **ens18** es ethernet en la ranura (slot) 18; en las máquinas virtuales de Proxmox suelen ser **ens18**, **ens19**... y en VirtualBox **enp0s3**, **enp0s8**...
- **Contenedor LXC de Proxmox** (nuestro caso): el nombre **lo decide Proxmox**. Es el campo *Name* de cada tarjeta en la pestaña *Network* del contenedor. Por defecto propone **eth0**, **eth1**..., pero se puede poner cualquier otro. En nuestro escenario se llaman **ens18** y **ens19**.

Sea cual sea el caso, comprobad siempre los nombres reales con **ip -br link** antes de configurar nada.

La orden **ifconfig** (paquete **net-tools**) está obsoleta y no viene instalada. Usamos siempre **ip**.

1.2. Configuración temporal con **ip**

Sirve para hacer pruebas rápidas. **Se pierde al reiniciar** el equipo o la red.

```

sudo ip link set ens19 up           # activar la interfaz
sudo ip address add 192.168.29.1/24 dev ens19 # añadir una IP
sudo ip address del 192.168.29.1/24 dev ens19 # quitarla
sudo ip route add default via 10.3.129.1      # puerta de enlace (la del
router del aula)
sudo ip address flush dev ens19              # borrar todas las IP de la
interfaz

```

1.3. Configuración permanente: `/etc/network/interfaces`

En nuestros servidores la red la gestiona **ifupdown** (servicio **networking**), y este es su fichero principal. Ejemplo para nuestro servidor, con una interfaz por DHCP hacia Internet y otra estática hacia la red local:

```

source /etc/network/interfaces.d/*

auto lo
iface lo inet loopback

allow-hotplug ens18
iface ens18 inet dhcp

auto ens19
iface ens19 inet static
    address 192.168.29.1/24

```

Cuando la interfaz estática es la única del equipo, hay que añadir también la puerta de enlace y los DNS. Por ejemplo, el cliente estático del esquema (apartado 1.9), que sale a Internet a través del servidor:

```

auto ens18
iface ens18 inet static
    address 192.168.29.20/24
    gateway 192.168.29.1
    dns-nameservers 1.1.1.1 8.8.8.8
    dns-search sri.local

```

Significado de cada línea:

Línea	Significado
auto ens19	activa la interfaz al arrancar el sistema.
allow-hotplug ens18	la activa cuando el sistema detecta la tarjeta. Es lo que pone el instalador de Debian. En la práctica, para tarjetas fijas funciona igual que auto .
iface ens19 inet static	configuración IPv4 (inet) estática. Con dhcp en lugar de static , la pide por DHCP. Para IPv6 se usa inet6 .
address 192.168.29.1/24	IP y máscara en notación CIDR. También vale la forma antigua con una línea netmask 255.255.255.0 aparte.
gateway	puerta de enlace. Solo debe haber una en todo el fichero.

Línea	Significado
dns-nameservers y dns-search	servidores DNS y dominio de búsqueda. Solo funcionan si está instalado el paquete resolvconf . Si no, se ignoran y hay que editar /etc/resolv.conf a mano (ver 1.4).
source /etc/network/interfaces.d/*	incluye también los ficheros de esa carpeta, por si se prefiere un fichero por interfaz.

Aplicar los cambios:

```
sudo ifdown ens19 && sudo ifup ens19      # reinicia solo esa interfaz
sudo systemctl restart networking        # reinicia todas
```

Cuidado: si estáis conectados por SSH, reiniciar la interfaz por la que entráis os puede dejar sin conexión si hay un error en el fichero. Es mejor tocar la configuración desde la consola del equipo en Proxmox.

Si **ifup** responde que la interfaz ya está configurada, primero hay que hacer **ifdown**. Si se editó el fichero con la interfaz levantada y **ifdown** falla, se puede limpiar con **sudo ip address flush dev ens19** y después hacer **ifup**.

«**ifup: orden no encontrada**» al entrar como root con **su: ifup, ifdown** y otras órdenes de administración están en **/usr/sbin**, que no está en el **PATH** de un usuario normal. Si os hacéis root con **su** (sin guion), se conserva el **PATH** del usuario y la orden «no existe». Se nota porque el prompt sigue en vuestra carpeta: **root@debian:/home/usuario#**. Soluciones: entrar con **su -** (carga el entorno completo de root), usar **sudo** o escribir la ruta completa (**/usr/sbin/ifdown ens19**).

Comprobar qué paquete gestiona la red: **dpkg -l ifupdown ifupdown2**. En Debian 12, tanto en máquina virtual como en contenedor LXC, aparece instalado **ifupdown** (versión 0.8.x). **ifupdown2** es la versión que usa el propio anfitrión Proxmox. Los dos leen **/etc/network/interfaces** y se manejan con **ifup/ifdown**.

Contenedores LXC de Proxmox (nuestro caso): en un contenedor LXC la red **la gestiona Proxmox**. Nombre, IP, puerta de enlace y DHCP de cada tarjeta se configuran desde la interfaz web (contenedor → Network). Proxmox **reescribe /etc/network/interfaces** al arrancar el contenedor, así que lo que editéis a mano se pierde al reiniciarlo. Tenéis dos opciones:

- Configurar la red desde Proxmox (lo recomendable): **ens18** con IPv4 *DHCP* y **ens19** con IPv4 *Static 192.168.29.1/24*, sin puerta de enlace. Después, abrid **/etc/network/interfaces** dentro del contenedor y veréis que Proxmox lo ha escrito igual que en el ejemplo de arriba.

- Editarlo a mano, creando antes el fichero vacío `/etc/network/.pve-ignore.interfaces` para que Proxmox no lo sobrescriba.
- Los DNS (`/etc/resolv.conf`) y el nombre del equipo (`/etc/hostname`) también los gestiona Proxmox en los contenedores (pestaña DNS y opciones del contenedor).

1.4. DNS: `/etc/resolv.conf`

Indica a qué servidores DNS pregunta el equipo:

```
search sri.local
nameserver 1.1.1.1
nameserver 8.8.8.8
```

- Si una interfaz recibe su configuración por **DHCP**, este fichero **lo sobrescribe el cliente DHCP**, así que no merece la pena editarlo a mano.
- Si toda la configuración es estática y no está instalado `resolvconf`, se edita directamente.

1.5. Nombre del equipo: `/etc/hostname` y `/etc/hosts`

```
sudo hostnamectl set-hostname server2    # cambia /etc/hostname
hostnamectl                               # comprobar
```

Después hay que actualizar la línea `127.0.1.1` de `/etc/hosts`, que es la que resuelve el nombre del propio equipo:

```
127.0.0.1    localhost
127.0.1.1    server2.sri.local server2
```

1.6. El servidor como router de la red local

En nuestro escenario, los clientes usan el servidor (`192.168.29.1`) como puerta de enlace. Para que puedan salir a Internet a través de él hay que hacer tres cosas: activar el reenvío de paquetes, permitir en el cortafuegos el tráfico reenviado y hacer NAT. En nuestro servidor el cortafuegos lo gestiona **UFW**, así que **toda la configuración se hace con UFW**: con la orden `ufw` y, cuando no hay orden para algo (el NAT y el reenvío), en los ficheros de configuración del propio UFW, dentro de `/etc/ufw/`.

Paso	Qué conseguimos	Cómo
1	Que el servidor reenvíe paquetes entre <code>ens19</code> y <code>ens18</code>	<code>net/ipv4/ip_forward=1</code> en <code>/etc/ufw/sysctl.conf</code> y <code>sudo ufw reload</code>
2	Que UFW deje pasar el tráfico reenviado de los clientes	<code>sudo ufw route allow in on ens19 out on ens18 from 192.168.29.0/24</code>

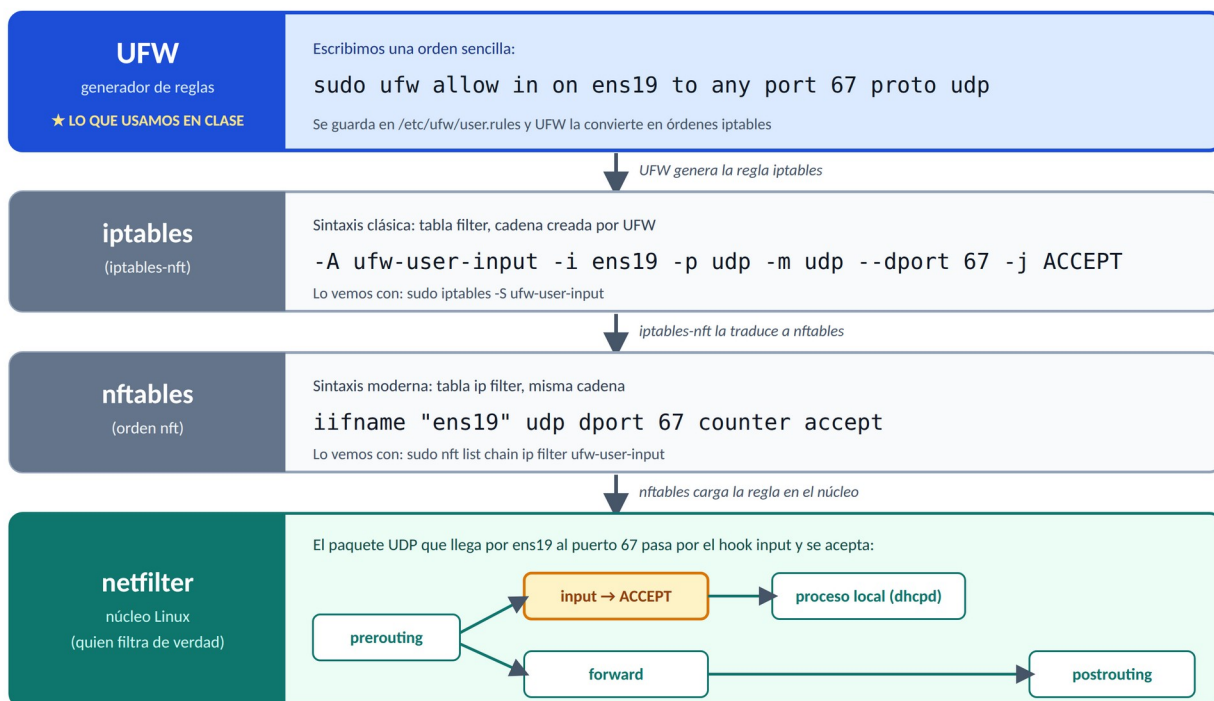
Paso	Qué conseguimos	Cómo
3	Que los clientes salgan a Internet con la IP de ens18 (NAT)	Bloque *nat en /etc/ufw/before.rules y sudo ufw reload
4	Que los clientes lleguen a los servicios del servidor (DHCP, DNS)	sudo ufw allow in on ens19 to any port 67 proto udp (y el 53 para DNS)

Cortafuegos en Linux: netfilter, iptables, nftables y UFW

Es fácil liarse, porque hay varias capas que se apilan unas sobre otras. La clave es separar **quién filtra de verdad** de **qué herramienta se usa para escribir las reglas**.

La misma regla, de la capa más alta a la más baja

Ejemplo: abrir el puerto del servidor DHCP (67/udp) en la interfaz interna ens19



La misma regla en cada capa

- **netfilter** es quien hace el trabajo: un subsistema del núcleo Linux que filtra, reenvía y hace NAT. Todo lo demás son formas de decirle qué reglas aplicar. Los *hooks* (**input**, **forward**, **postrouting**...) son los puntos de netfilter por los que pasan los paquetes.
- **iptables** es la herramienta clásica. Tiene tablas y cadenas fijas (**filter**, **nat**, **INPUT**, **FORWARD**, **POSTROUTING**...) y una orden distinta para cada protocolo: **iptables**, **ip6tables**, **arptables**, **ebtables**.
- **nftables** (la orden **nft**) es su sustituto moderno, y el que usa Debian 12 por debajo. Hace lo mismo con un único lenguaje para IPv4, IPv6 y más.
- **iptables-nft** es el puente entre ambos. Para no romper los programas que usan iptables, Debian instala una orden **iptables** con la sintaxis de siempre que, por debajo, crea reglas de nftables.

- **UFW** (*Uncomplicated Firewall*) no es un cortafuegos en sí, sino un **generador de reglas**. Escribimos `ufw allow 22` y UFW crea por nosotros muchas cadenas (`ufw-before-input`, `ufw-user-input`...) usando órdenes `iptables`, que en Debian 12 pasan por `iptables-nft` y acaban como reglas de `nftables`.

Para saber qué tenéis:

```
sudo ufw status verbose      # si dice "inactive", UFW no interviene
iptables -V                 # "(nf_tables)" = iptables-nft · "(legacy)" =
el antiguo                  # herramienta para consultar las reglas
nft --version
```

Qué significa en la práctica. Todo acaba en el mismo netfilter, así que un paquete pasa por **todas** las cadenas enganchadas a ese *hook*, y **basta con que una lo descarte para que se pierda**. Por eso hay que **elegir una sola herramienta como «dueña» del cortafuegos**. En nuestro servidor es **UFW**:

- Todas las reglas (permitir, denegar, reenvío y NAT) se crean **con UFW** o en sus ficheros (`/etc/ufw/`).
- **No añadáis reglas a mano con `nft` ni con `iptables`**: UFW no las conoce y las borra o las duplica al recargar.
- `nft` e `iptables -L` se usan **solo para mirar y depurar** lo que ha generado UFW.

Podéis seguir la regla del esquema por todas las capas en vuestro servidor. Primero la creamos con UFW y después vemos lo que ha generado en cada nivel:

```
sudo ufw allow in on ens19 to any port 67 proto udp  # 1. UFW
sudo iptables -S ufw-user-input                     # 2. iptables
sudo nft list chain ip filter ufw-user-input          # 3. nftables

-A ufw-user-input -i ens19 -p udp -m udp --dport 67 -j ACCEPT

table ip filter {
    chain ufw-user-input {
        iifname "ens19" udp dport 67 counter packets 0 bytes 0 accept
    }
}
```

La regla es la misma en las tres capas; solo cambia cómo se escribe. Al final, netfilter la aplica en el *hook* `input` a cada paquete UDP que llega por `ens19` al puerto 67.

Importante: en este módulo vamos a trabajar con UFW. Es el cortafuegos que tiene nuestro servidor y el más sencillo de manejar. `iptables` y `nftables` los veréis solo para entender qué hace UFW por debajo y para consultar las reglas, nunca para crearlas.

Si UFW no está instalado: `sudo apt install ufw`. **Antes de activarlo** permitid el acceso SSH, o perderéis la conexión si administráis el servidor por SSH: `sudo ufw allow OpenSSH` (o `sudo ufw allow 22/tcp`) y después `sudo ufw enable`. `ufw enable` también deja UFW activado para los siguientes arranques.

Para saber más

- [Uncomplicated Firewall \(ufw\)](#), en la [wiki de Debian en español](#): instalación y órdenes básicas.
- [Firewall](#), en la [documentación oficial de Ubuntu Server](#) (en inglés): UFW, reenvío y NAT con `/etc/ufw/before.rules`, como en estos apuntes.
- El manual del propio sistema: `man ufw` y `man ufw-framework`. El segundo explica para qué sirve cada fichero de `/etc/ufw/`.

Paso 1. Activar el reenvío de paquetes IPv4

Por defecto Linux no reenvía paquetes de una interfaz a otra. UFW tiene su propio fichero de parámetros del núcleo, `/etc/ufw/sysctl.conf`, que aplica cada vez que arranca o se recarga. La línea del reenvío ya viene escrita, pero comentada: basta con quitarle el `#`.

```
sudo nano /etc/ufw/sysctl.conf      # quitar el # de: #net/ipv4/ip_forward=1
sudo ufw reload                     # aplicarlo sin reiniciar
sysctl net.ipv4.ip_forward          # debe mostrar = 1
```

En este fichero los parámetros se escriben con `/` en lugar de `.`: `net/ipv4/ip_forward` es la misma variable que `net.ipv4.ip_forward`. Solo se aplica si UFW está activado (`sudo ufw enable`). En nuestras pruebas el valor pasó de 0 a 1 con `ufw reload`, y también al desactivar y volver a activar UFW. Si en algún fichero de `/etc/sysctl.d/` hay otro valor para `ip_forward`, quitadlo para no tener dos sitios que se contradigan.

Paso 2. Permitir el tráfico reenviado

UFW trae la política de reenvío en **denegar** («deny (routed)» en `ufw status verbose`): aunque el reenvío y el NAT estén bien, **bloquea el tráfico de los clientes**. Permitimos solo el que va de la red interna hacia fuera:

```
sudo ufw route allow in on ens19 out on ens18 from 192.168.29.0/24
```

Se lee así: permite reenviar (`route allow`) lo que **entra** por `ens19` (la red interna) y **sale** por `ens18` (hacia el aula), si viene de la red `192.168.29.0/24`. Las respuestas que vuelven se permiten solas, porque UFW acepta el tráfico de las conexiones ya establecidas.

Cuidado con ping al comprobarlo. UFW deja pasar siempre los `ping` reenviados (su fichero `/etc/ufw/before.rules` acepta ICMP en la cadena de reenvío), **aunque no exista esta regla**. En nuestras pruebas, sin la regla `route allow` el `ping` funcionaba pero las conexiones TCP (web, `apt`...) no. Para comprobar el cortafuegos usad una conexión TCP, por ejemplo `curl -I http://deb.debian.org` o `sudo apt update` desde el cliente.

Otra opción es cambiar la política por defecto para aceptar todo el tráfico reenviado: `sudo ufw default allow routed`. Es más cómodo pero menos seguro; la regla `route allow` es más precisa.

Paso 3. Hacer NAT (enmascaramiento)

`ufw` no tiene una orden para el NAT: la regla se escribe en `/etc/ufw/before.rules`, con la sintaxis de `iptables`. Guardamos antes una copia y añadimos este bloque **al principio del fichero, antes de la línea `*filter`**:

```
sudo cp /etc/ufw/before.rules /etc/ufw/before.rules.orig
sudo nano /etc/ufw/before.rules

*nat
:POSTROUTING ACCEPT [0:0]
-F POSTROUTING
-A POSTROUTING -s 192.168.29.0/24 -o ens18 -j MASQUERADE
COMMIT

*filter
...
```

Qué hace cada línea:

Línea	Qué hace
<code>*nat ... COMMIT</code>	todo lo que va entre ellas son reglas de la tabla <code>nat</code> .
<code>:POSTROUTING ACCEPT [0:0]</code>	la cadena <code>POSTROUTING</code> , que se aplica justo antes de que el paquete salga del servidor , cuando ya se ha decidido por qué interfaz sale. Por defecto deja pasar los paquetes sin cambios.
<code>-F POSTROUTING</code>	vacía la cadena antes de añadir la regla. Sin esta línea, cada <code>ufw reload</code> añade otra copia de la regla (lo comprobamos en las pruebas: tras tres recargas había tres reglas iguales).
<code>-A POSTROUTING</code>	añade una regla a esa cadena.
<code>-s 192.168.29.0/24</code>	solo para paquetes que vienen de la red interna de los clientes.
<code>-o ens18</code>	y que salen por <code>ens18</code> , la interfaz hacia la red del aula.
<code>-j MASQUERADE</code>	cambia la IP de origen del cliente por la IP que tenga <code>ens18</code> en ese momento. Así el router del aula ve todo el tráfico como si viniera del servidor y sabe devolver las respuestas; cuando vuelven, el servidor deshace el cambio y se las entrega al cliente.

Se usa `MASQUERADE` y no `SNAT` porque `ens18` recibe su IP por DHCP y puede cambiar: `MASQUERADE` usa siempre la IP actual de la interfaz, mientras que `SNAT` necesita una IP fija escrita en la regla.

Aplicamos los cambios:

```
sudo ufw reload
```

Editar el fichero **no carga nada por sí solo**: hay que hacer `ufw reload` cada vez que se modifica. Si hay un error de sintaxis en el fichero, `ufw reload` muestra un error.

En las pruebas, un cliente de la red interna solo consiguió conexión TCP con un equipo de la red externa cuando estaban los tres pasos. Sin NAT, el equipo de fuera recibía los paquetes con la IP `192.168.29.20` del cliente y no sabía cómo devolver la respuesta.

Paso 4. Abrir los servicios del propio servidor (DHCP, DNS)

Lo que va dirigido al propio servidor (por ejemplo, las peticiones DHCP o DNS de los clientes) es tráfico **de entrada**, no de reenvío, así que `ufw route allow` no lo cubre. Hay que abrirlo aparte, solo en la interfaz interna:

```
sudo ufw allow in on ens19 to any port 67 proto udp    # DHCP
sudo ufw allow in on ens19 to any port 53             # DNS (UDP y TCP)
```

En nuestras pruebas, `isc-dhcp-server` siguió repartiendo IPs con UFW activo y sin esta regla, porque recibe los paquetes DHCP con un socket de bajo nivel que no pasa por el cortafuegos (Kea hace lo mismo con `"dhcp-socket-type": "raw"`). En cambio, `dnsmasq` no dio ninguna IP hasta abrir el puerto 67, y un servidor DNS siempre necesita el 53 abierto. Abrid los puertos de todas formas: así no dependéis de cómo reciba los paquetes cada programa.

Ver y borrar reglas

Las reglas creadas con la orden `ufw` se ven y se borran con la propia `ufw`:

```
sudo ufw status verbose          # estado, políticas por defecto y reglas
sudo ufw status numbered        # reglas numeradas
sudo ufw delete 3               # borra la regla número 3 (pide confirmación)
```

	To	Action	From
	--	-----	----
[1]	OpenSSH	ALLOW IN	Anywhere
[2]	Anywhere on ens18	ALLOW FWD	192.168.29.0/24 on ens19

Las reglas de `/etc/ufw/before.rules` (como el NAT) **no aparecen en `ufw status`**. Para verlas, y para depurar en general, se consulta lo que UFW ha cargado en el núcleo:

```
sudo iptables -t nat -L POSTROUTING -n -v    # tabla nat con contadores de paquetes
sudo nft list tables                          # tablas existentes
sudo nft list table ip nat                    # la tabla nat en sintaxis de nftables
sudo nft list ruleset                        # todo lo cargado

Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target      prot opt in     out     source
destination
  1      84 MASQUERADE  0    --  *      ens18  192.168.29.0/24      0.0.0.0/0
```

La columna `pkts` cuenta los paquetes que han pasado por la regla: si al navegar desde un cliente no aumenta, el tráfico no está llegando a la regla.

Al listar con `nft` veréis tablas `ip filter` o `ip nat` con cadenas en mayúsculas y cadenas `ufw-...`, además del aviso `# Warning: table ip nat is managed by iptables-nft, do not touch!`. Son las reglas de UFW traducidas a nftables. **Solo para mirar:** para cambiar algo, usad `ufw` o sus ficheros y después `sudo ufw reload`.

1.7. Cómo dejamos los cambios permanentes

Un cambio es **permanente** si sigue ahí después de reiniciar el servidor. Con UFW casi todo lo es desde el primer momento, porque cada orden se guarda en un fichero. Con **iptables** a mano, las reglas solo están en memoria y **se pierden al reiniciar** si no las guardamos.

En clase usamos la primera columna. La segunda es para que sepáis cómo se hace sin UFW, porque la encontraréis en muchos tutoriales.

Si lo hemos configurado con UFW	Si lo hemos configurado con iptables
Reglas de filtrado. <code>sudo ufw allow ...</code> y <code>sudo ufw route allow ...</code> se guardan solas en <code>/etc/ufw/user.rules</code> . No hay que hacer nada más.	Reglas de filtrado. <code>sudo iptables -A ...</code> solo cambia la memoria. Hay que guardarlas con <code>sudo netfilter-persistent save</code> (paquete <code>iptables-persistent</code>), que las escribe en <code>/etc/iptables/rules.v4</code> .
NAT. El bloque <code>*nat</code> de <code>/etc/ufw/before.rules</code> ya está en un fichero, así que es permanente. <code>sudo ufw reload</code> lo aplica sin reiniciar.	NAT. <code>sudo iptables -t nat -A POSTROUTING -s 192.168.29.0/24 -o ens18 -j MASQUERADE</code> y después <code>sudo netfilter-persistent save</code> .
Reenvío IPv4. La línea <code>net/ipv4/ip_forward=1</code> descomentada en <code>/etc/ufw/sysctl.conf</code> . UFW la aplica cada vez que arranca y con <code>sudo ufw reload</code> .	Reenvío IPv4. <code>sudo sysctl -w net.ipv4.ip_forward=1</code> es temporal. Para que dure: <code>net.ipv4.ip_forward=1</code> en <code>/etc/sysctl.d/99-router.conf</code> y <code>sudo sysctl --system</code> .
Políticas por defecto. <code>sudo ufw default deny incoming</code> , <code>sudo ufw default deny routed...</code> se guardan en <code>/etc/default/ufw</code> .	Políticas por defecto. <code>sudo iptables -P FORWARD DROP</code> y después <code>sudo netfilter-persistent save</code> .
Cargar todo al arrancar. <code>sudo ufw enable</code> deja <code>ENABLED=yes</code> en <code>/etc/ufw/ufw.conf</code> , y el servicio <code>ufw</code> carga las reglas en cada arranque.	Cargar todo al arrancar. El servicio <code>netfilter-persistent</code> , que se activa al instalar el paquete, carga <code>/etc/iptables/rules.v4</code> en cada arranque.
Comprobar tras reiniciar. <code>sudo ufw status verbose</code> y <code>sysctl net.ipv4.ip_forward</code> .	Comprobar tras reiniciar. <code>sudo iptables -S</code> , <code>sudo iptables -t nat -S</code> y <code>sysctl net.ipv4.ip_forward</code> .

Importante: no mezcléis las dos columnas. El paquete `ufw` no puede instalarse junto a `iptables-persistent` (`apt-cache show ufw` lo indica en la línea *Breaks*), así que al instalar uno apt desinstala el otro. Además, UFW no conoce las reglas hechas a mano con `iptables` y al recargar las pierde o las duplica. En nuestro servidor usamos **solo UFW**.

1.8. Comprobaciones

<code>ip -br address</code>	<code># ¿tiene cada interfaz la IP correcta?</code>
<code>ip route</code>	<code># ¿hay ruta por defecto?</code>

```

ping -c 3 192.168.29.1      # conectividad en la red local
ping -c 3 1.1.1.1          # salida a Internet (sin DNS)
ping -c 3 deb.debian.org   # resolución DNS
curl -I http://deb.debian.org # conexión TCP (web); si falta: sudo apt
install curl
getent hosts deb.debian.org  # qué IP resuelve
ip route get 1.1.1.1         # por qué interfaz y puerta de enlace saldría
tracpath -n 1.1.1.1         # cada salto del camino (o: traceroute -n
1.1.1.1)

```

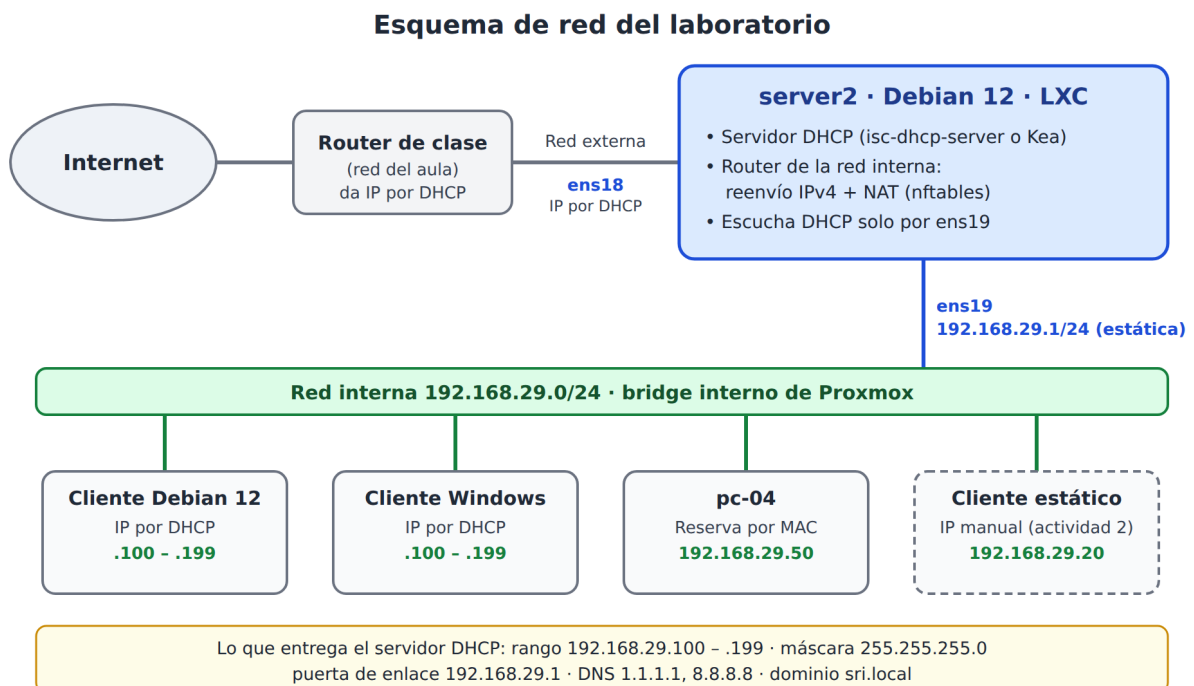
1.1.1.1 es el servidor DNS público de Cloudflare. Lo usamos para probar la salida a Internet **sin depender del DNS**, porque es una IP y no hace falta resolver ningún nombre:

- Si falla **ping 1.1.1.1**, el problema está en la **ruta, la puerta de enlace, el reenvío o el NAT**.
- Si **ping** funciona pero no se puede navegar ni hacer **apt update** desde un cliente, revisad la regla **ufw route allow** del apartado 1.6: UFW deja pasar los **ping** reenviados aunque bloquee el resto. Para comprobarlo, usad una conexión TCP: **curl -I http://deb.debian.org**.
- Si funciona **ping 1.1.1.1** pero falla **ping deb.debian.org**, el problema es el **DNS**.

En nuestro esquema, el ping de un cliente sigue este camino: **cliente → 192.168.29.1 (el ens19 del servidor) → NAT por ens18 → router del aula → Internet**. Desde un cliente, **ip route get 1.1.1.1** debe mostrar **via 192.168.29.1**, y **tracpath** debe tener el servidor como primer salto. Si **tracpath** no está instalado: **sudo apt install iputils-tracpath** (o **traceroute**).

1.9. Esquema de red que vamos a montar

Esta es la red que montaremos en Proxmox. En las actividades 1 y 2 dejamos preparada la red del servidor y de un cliente; en los apartados siguientes montamos el servicio DHCP sobre ella.



Esquema de red del laboratorio

Equipo	Interfaz	Red en Proxmox	Configuración
server2 (Debian 12)	ens18	Bridge externo (red del aula)	DHCP, recibida del router
server2 (Debian 12)	ens19	Bridge interno	Estática 192.168.29.1/24
Cliente Debian 12	ens18	Bridge interno	DHCP (rango .100-.199)
Cliente Windows	Ethernet	Bridge interno	DHCP (rango .100-.199)
pc-04	ens18	Bridge interno	DHCP con reserva 192.168.29.50
Cliente estático	ens18	Bridge interno	Manual 192.168.29.20/24, puerta de enlace 192.168.29.1

En Proxmox, la segunda tarjeta del servidor y la tarjeta de todos los clientes deben estar conectadas al mismo bridge interno; si no, no se verán entre ellos. El servidor (192.168.29.1) es su puerta de enlace, así que los clientes solo saldrán a Internet cuando el servidor tenga activados el reenvío y el NAT (apartado 1.6).

ACTIVIDAD 1: configura tu servidor con ens18 por DHCP y ens19 estática en 192.168.29.1/24 (si es un contenedor LXC, desde la pestaña Network de Proxmox, ver 1.3), cambia su nombre a server2 y comprueba la configuración con las órdenes de 1.8. Revisa qué ha quedado escrito en /etc/network/interfaces. Reinicia la máquina y verifica que todo se mantiene.

ACTIVIDAD 2: configura un cliente con IP estática 192.168.29.20/24 y puerta de enlace 192.168.29.1. Activa el reenvío y el NAT en el servidor, y comprueba que el cliente sale a Internet. ¿Qué pasa si desactivas el reenvío durante un momento con `sudo sysctl -w net.ipv4.ip_forward=0`? (Es un cambio temporal; vuelve a activarlo con `sudo ufw reload`.)

ACTIVIDAD 3: con el cliente estático navegando (`curl -I http://deb.debian.org`), haz estas pruebas en el servidor y anota qué pasa en cada una:

1. Mira la regla de NAT con `sudo iptables -t nat -L POSTROUTING -n -v` y comprueba que su contador de paquetes aumenta.
2. Borra la regla de reenvío (`sudo ufw status numbered` y `sudo ufw delete N`). ¿Funciona el `ping 1.1.1.1` desde el cliente? ¿Y el `curl`? ¿Por qué?

3. Vuelve a crearla. Comenta con `#` la línea `-A POSTROUTING ... MASQUERADE` de `/etc/ufw/before.rules` y haz `sudo ufw reload`. ¿Qué pasa ahora con el `ping` y el `curl`?
4. Déjalo todo como estaba y reinicia el servidor. Comprueba con las órdenes de la tabla de 1.7 que las reglas y el reenvío siguen activos.

2. Antes de empezar: ¿qué software DHCP usamos?

El servidor DHCP «clásico» de Linux es **ISC DHCP** (paquete `isc-dhcp-server`), desarrollado por el Internet Systems Consortium. **ISC dejó de mantenerlo a finales de 2022** y recomienda su sucesor, **Kea** (paquetes `kea-dhcp4-server` y `kea-dhcp6-server`).

Debian 12 (Bookworm) todavía incluye `isc-dhcp-server` (versión 4.4.3), así que en estos apuntes lo usaremos para aprender los conceptos, porque su configuración es muy legible. En el **Anexo A** tenéis la misma configuración hecha con Kea, que es lo que encontraréis cada vez más en entornos reales.

	ISC DHCP	Kea
Paquete en Debian 12	<code>isc-dhcp-server</code>	<code>kea-dhcp4-server</code>
Estado	Sin mantenimiento desde 2022	Mantenido por ISC
Fichero de configuración	<code>/etc/dhcp/dhcpd.conf</code>	<code>/etc/kea/kea-dhcp4.conf</code> (JSON)
Concesiones	<code>/var/lib/dhcp/dhcpd.leases</code>	<code>/var/lib/kea/kea-leases4.csv</code>
Servicio systemd	<code>isc-dhcp-server</code>	<code>kea-dhcp4-server</code>

3. Escenario

Nuestro servidor, `server2`, tiene dos interfaces de red:

- `ens18`: dinámica, recibe su configuración del router que da salida a Internet.
- `ens19`: estática, conectada a la red local interna `192.168.29.0/24`, con la IP `192.168.29.1`.

El servicio DHCP escuchará **solo** por la interfaz interna, que **debe tener IP estática** antes de instalar nada. Configura la red del servidor tal como se explica en el apartado 1.3 y comprobada con las órdenes de 1.8.

4. Instalación

Actualizamos la lista de paquetes e instalamos el servidor:

```
sudo apt update
sudo apt install isc-dhcp-server
```

Al terminar, la instalación intenta arrancar el servicio y **falla**. Es normal: todavía no lo hemos configurado. Vemos por qué con:

```
sudo systemctl status isc-dhcp-server
sudo journalctl -u isc-dhcp-server -e
```

Cambio importante respecto a versiones antiguas: Debian 12 ya no instala **rsyslog** por defecto, así que no existe **/var/log/syslog**. Los registros se consultan con **journalctl**, que los filtra por servicio (**-u**), muestra el final (**-e**) o los sigue en directo (**-f**).

ACTIVIDAD 4: revisa con **journalctl -u isc-dhcp-server** los mensajes del servicio. ¿Qué errores aparecen? Coméntalo con tus compañeros.

En el registro veremos, principalmente, dos problemas:

- **No se ha indicado la interfaz** por la que el servicio debe escuchar, así que intenta arrancar en todas y además lanza a la vez los servidores de IPv4 e IPv6.
- **No hay ninguna declaración subnet** (ámbito) que coincida con las redes de las interfaces: **No subnet declaration for ens19 (192.168.29.1)**.

4.1. Interfaz de escucha

Se indica en **/etc/default/isc-dhcp-server**. En versiones actuales hay **una variable para IPv4 y otra para IPv6** (la antigua **INTERFACES** ya no se usa):

```
INTERFACESv4="ens19"
INTERFACESv6=""
```

Si hay varias interfaces, se separan con espacios: **INTERFACESv4="ens19 ens20"**. Dejando **INTERFACESv6** vacía evitamos que intente arrancar también el servidor DHCPv6.

5. Configuración: /etc/dhcp/dhcpd.conf

El fichero que instala el paquete trae muchos ejemplos comentados que podéis usar como referencia. Conviene guardar una copia antes de editarlo:

```
sudo cp /etc/dhcp/dhcpd.conf /etc/dhcp/dhcpd.conf.orig
```

Ejemplo completo para nuestro escenario:

```
authoritative;

default-lease-time 600;          # 10 minutos
max-lease-time 7200;            # 2 horas
option domain-name "sri.local";
```

```
option domain-name-servers 1.1.1.1, 8.8.8.8;

subnet 192.168.29.0 netmask 255.255.255.0 {
    range 192.168.29.100 192.168.29.199;
    option routers 192.168.29.1;
    option broadcast-address 192.168.29.255;
}

host pc-04 {
    hardware ethernet bc:24:11:4c:7f:1d;
    fixed-address 192.168.29.50;
}
```

Parámetros más importantes:

Parámetro	Para qué sirve
authoritative	el servidor se considera el responsable de esa red. Sin él, un cliente que venga de otra red con una IP antigua tardará más en obtener una nueva.
default-lease-time	duración de la concesión, en segundos, si el cliente no pide una concreta.
max-lease-time	duración máxima que se concede aunque el cliente pida más.
option domain-name	nombre de dominio que se entrega a los clientes.
option domain-name-servers	servidores DNS, separados por comas .
subnet ... netmask ...	declara un ámbito: la dirección de red y su máscara.
range	intervalo de IP que se reparten dinámicamente. Tiene que estar dentro de la red del subnet . Puede haber varias líneas range .
option routers	puerta de enlace (gateway) para los clientes.
option broadcast-address	dirección de difusión (broadcast) de la red. Ojo: no es lo mismo que multidifusión (multicast).
option subnet-mask	máscara que se entrega al cliente. Si no se pone, se usa la del subnet , así que normalmente no hace falta.
host	define una reserva . El nombre es solo una etiqueta.
hardware ethernet	MAC del equipo, escrita con dos puntos (bc:24:11:4c:7f:1d). El formato de Windows con guiones (BC-24-11-4C-7F-1D) da error.
fixed-address	IP reservada. Se recomienda que esté fuera del range para no chocar con las dinámicas.
server-identifier (opcional)	IP con la que el servidor se identifica ante los clientes. Si el servidor tiene varias interfaces, se usa la de la interfaz por la que responde, así que casi nunca hace falta.

Errores frecuentes: comillas tipográficas ("sri.com") copiadas desde un procesador de textos en lugar de comillas rectas ("sri.com"); olvidar un ;; poner un punto final en una IP (8.8.8.8.); rangos que se salen de la red del subnet.

5.1. Comprobar la sintaxis antes de reiniciar

Antes de reiniciar el servicio, comprobamos que el fichero es correcto:

```
sudo dhcpd -t -cf /etc/dhcp/dhcpd.conf
```

Si hay errores, indica la línea y marca con ^ dónde está el problema. Por ejemplo:

```
/etc/dhcp/dhcpd.conf line 3: semicolon expected.  
    option domain-name-servers 8.8.8.8.  
                                ^  
/etc/dhcp/dhcpd.conf line 8: expecting numeric value.  
    hardware ethernet 00-08-74-4C-7F-1D;  
                        ^  
Configuration file errors encountered -- exiting
```

ACTIVIDAD 5: introduce a propósito los dos errores anteriores en tu `dhcpd.conf` y comprueba qué te dice `dhcpd -t`. Después corrígelos.

6. Arrancar y parar el servicio

Los servicios se gestionan con **systemctl** (las órdenes antiguas `/etc/init.d/...` y `service ...` siguen funcionando por compatibilidad, pero ya no son la forma recomendada):

```
sudo systemctl start    isc-dhcp-server  
sudo systemctl stop     isc-dhcp-server  
sudo systemctl restart  isc-dhcp-server  
sudo systemctl status   isc-dhcp-server  
sudo systemctl enable   isc-dhcp-server # arrancar al iniciar el sistema
```

Cada vez que cambiemos la configuración hay que reiniciar el servicio (no el servidor completo): `sudo systemctl restart isc-dhcp-server`.

Para ver en directo lo que hace el servidor mientras los clientes piden IP:

```
sudo journalctl -u isc-dhcp-server -f
```

Veremos el intercambio **DORA**: `DHCPDISCOVER`, `DHCPOFFER`, `DHCPREQUEST` y `DHCPACK`.

Si el servicio está activo pero los clientes no reciben IP, revisad en este orden:

1. Que **INTERFACESv4** sea la interfaz interna (`ens19`) y que tenga su IP estática (`ip -br address`).

2. Que haya un **subnet** para la red de esa interfaz y que el **range** esté dentro de ella (`sudo dhcpcd -t -cf /etc/dhcp/dhcpd.conf`).
3. Que los clientes estén en el mismo bridge interno de Proxmox que **ens19**.
4. El registro en directo (`sudo journalctl -u isc-dhcp-server -f`) y el tráfico (`sudo tcpdump -ni ens19 port 67 or port 68`, ver 7.3): si no aparece ningún **DHCPDISCOVER**, las peticiones no llegan al servidor.
5. El cortafuegos: si hay UFW activo, abrid el puerto 67/udp en **ens19** (ver el aviso de UFW en el apartado 1.6).

ACTIVIDAD 6: ejecuta `systemctl status isc-dhcp-server` y analiza la información que muestra: estado, PID, desde cuándo está activo y últimas líneas del registro.

7. Comprobación desde los clientes

7.1. Cliente Linux (Debian 12)

Con **ifupdown** y la interfaz configurada como **dhcp**:

```
sudo dhclient -r -v ens18    # libera la concesión actual (DHCPRELEASE)
sudo dhclient -v ens18      # pide una nueva (se ve el DORA completo)
ip -br address              # IP recibida
ip route                    # puerta de enlace
cat /etc/resolv.conf        # DNS y dominio
```

7.2. Cliente Windows 10/11

```
ipconfig /release
ipconfig /renew
ipconfig /all
```

ANALIZA: ejecuta `ipconfig /all` en un cliente Windows y localiza: IP, máscara, puerta de enlace, servidor DHCP, servidores DNS, sufijo DNS y fechas de concesión obtenida y de caducidad. ¿Cuadran con lo que configuraste en el servidor?

7.3. Ver el tráfico DHCP con **tcpdump**

tcpdump captura los paquetes que pasan por una interfaz y los muestra en el terminal. Es la forma más directa de **ver el DORA** y de saber si las peticiones de los clientes llegan al servidor. Se instala con `sudo apt install tcpdump` y se ejecuta como root. Se para con **Ctrl+C**.

Órdenes básicas, lanzadas en el servidor sobre la interfaz interna:

```
sudo tcpdump -D              # interfaces disponibles
sudo tcpdump -ni ens19       # todo el tráfico de ens19
sudo tcpdump -ni ens19 port 67 or port 68    # solo DHCP
sudo tcpdump -ni ens19 -v port 67 or port 68  # DHCP con todas las
```

```

opciones
sudo tcpdump -ni ens19 -e port 67 or port 68      # añade las MAC de origen y
destino
sudo tcpdump -ni ens19 -c 4 port 67 or port 68    # se para solo tras 4
paquetes
sudo tcpdump -ni ens19 ether host bc:24:11:4c:7f:1d  # solo el tráfico de un
equipo (por MAC)
sudo tcpdump -ni ens19 host 192.168.29.100        # solo el tráfico de una IP

```

Opción	Significado
-n	no traduce IPs ni puertos a nombres (más rápido y más claro).
-i ens19	interfaz en la que se escucha. Sin -i , tcpdump elige una por su cuenta.
port 67 or port 68	filtro. El servidor DHCP escucha en el puerto 67/udp y el cliente en el 68/udp .
-v	más detalle. En DHCP muestra el tipo de mensaje y las opciones (IP ofrecida, máscara, puerta de enlace, DNS, tiempo de concesión...).

Con el filtro de DHCP, al pedir una IP desde un cliente veremos algo así (el primer y el tercer paquete salen del cliente; el segundo y el cuarto, del servidor):

```

IP 0.0.0.0.68 > 255.255.255.255.67: BOOTP/DHCP, Request from
62:14:02:b5:c3:57, length 300
IP 192.168.29.1.67 > 192.168.29.100.68: BOOTP/DHCP, Reply, length 300
IP 0.0.0.0.68 > 255.255.255.255.67: BOOTP/DHCP, Request from
62:14:02:b5:c3:57, length 300
IP 192.168.29.1.67 > 192.168.29.100.68: BOOTP/DHCP, Reply, length 300

```

DHCP funciona sobre el antiguo protocolo BOOTP, por eso **tcpdump** solo dice *Request* o *Reply*. Para ver qué mensaje DORA es cada uno, añadimos **-v**. Si solo queremos las líneas importantes, podemos filtrar la salida con **grep** (**-l** hace que **tcpdump** escriba cada línea en cuanto la captura):

```

sudo tcpdump -lni ens19 -v port 67 or port 68 | grep -E "DHCP-Message|Your-IP"

    DHCP-Message (53), length 1: Discover
Your-IP 192.168.29.100
    DHCP-Message (53), length 1: Offer
    DHCP-Message (53), length 1: Request
Your-IP 192.168.29.100
    DHCP-Message (53), length 1: ACK

```

Fijaos en que:

- El cliente todavía no tiene IP, así que envía desde **0.0.0.0** a la dirección de difusión **255.255.255.255**.
- El servidor responde desde **192.168.29.1** y el campo **Your-IP** es la IP que ofrece al cliente.

- Al liberar la concesión (`dhclient -r`), el cliente ya tiene IP y envía un único paquete directamente al servidor (`192.168.29.100.68 > 192.168.29.1.67`).

Guardar la captura y abrirla con Wireshark. Con `-w` se guarda en un fichero en lugar de mostrarse, y con `-r` se vuelve a leer:

```
sudo tcpdump -ni ens19 -w /tmp/dhcp.pcap port 67 or port 68 # guardar
(Ctrl+C para parar)
tcpdump -nr /tmp/dhcp.pcap # leer el
fichero
tcpdump -nr /tmp/dhcp.pcap -v # leerlo con
detalle
```

El fichero `.pcap` se puede copiar al equipo del alumno (por ejemplo, con `scp`) y abrir con **Wireshark**, que muestra cada campo del paquete de forma gráfica. En Wireshark, el filtro de visualización para DHCP es `dhcp`.

Si no aparece ningún paquete al pedir IP desde un cliente, las peticiones no llegan al servidor: revisad que el cliente esté en el mismo bridge interno que `ens19`. Si aparecen los **Request** del cliente pero no los **Reply** del servidor, el problema está en el servidor DHCP (configuración, `INTERFACESv4` o cortafuegos).

ACTIVIDAD 7: deja `tcpdump -ni ens19 -v port 67 or port 68` capturando en el servidor y pide una IP desde un cliente Debian (`dhclient -r` y `dhclient -v`). Identifica los cuatro mensajes DORA y, en cada uno, las IP y puertos de origen y destino. ¿Qué opciones pide el cliente en el **Discover** (**Parameter-Request**) y cuáles le entrega el servidor en el **ACK**? Repite la captura guardándola con `-w` y ábrela en Wireshark.

8. Información de concesiones (leases)

El servidor guarda las concesiones que va asignando en `/var/lib/dhcp/dhcpd.leases`:

```
lease 192.168.29.100 {
  starts 1 2026/09/28 15:34:51;
  ends 1 2026/09/28 15:44:51;
  cltt 1 2026/09/28 15:34:51;
  binding state active;
  next binding state free;
  rewind binding state free;
  hardware ethernet c6:4a:69:5f:32:e3;
  client-hostname "cliente1";
}
```

Las fechas de este fichero están en **UTC**, así que en España veréis una o dos horas de diferencia con la hora local.

Para verlas en forma de tabla, el paquete incluye la herramienta `dhcp-lease-list`:

```
sudo dhcp-lease-list
```

ACTIVIDAD 8: libera la concesión de un cliente (`dhclient -r` o `ipconfig /release`) y comprueba qué cambia en `dhcpd.leases`. ¿Qué valor toma `binding state`?

Las **reservas** (`host`) no aparecen en este fichero, porque no son concesiones dinámicas.

9. Varias redes y pools

9.1. Un servidor con varias interfaces

Si el servidor está conectado a varias redes, basta con declarar un **subnet** por red y añadir todas las interfaces a `INTERFACESv4`:

```
INTERFACESv4="ens19 ens20"

subnet 192.168.10.0 netmask 255.255.255.0 {
    range 192.168.10.50 192.168.10.100;
    option routers 192.168.10.1;
    option domain-name-servers 8.8.8.8, 1.1.1.1;
}

subnet 192.168.20.0 netmask 255.255.255.0 {
    range 192.168.20.50 192.168.20.100;
    option routers 192.168.20.1;
    option domain-name-servers 8.8.8.8, 1.1.1.1;
}
```

El servidor sabe qué ámbito usar según la interfaz por la que le llega la petición.

9.2. Redes a las que el servidor no está conectado: agente relay

Las peticiones DHCP son **difusiones (broadcast)** y los routers **no las reenvían**. Para dar servicio a una red remota, el router de esa red debe tener un **agente de retransmisión (DHCP relay)**, que reenvía las peticiones al servidor en unidifusión. En Linux lo proporciona el paquete `isc-dhcp-relay`; en routers Cisco es la orden `ip helper-address`. En el servidor solo hay que declarar el **subnet** de esa red remota.

9.3. shared-network

shared-network no sirve para agrupar redes distintas cualquiera. Se usa cuando **varias subredes IP comparten el mismo segmento físico** (por ejemplo, se ha quedado pequeña la red y se añade una segunda subred en el mismo cable o VLAN). Todas las subredes de dentro se tratan como una sola red:

```
shared-network aulas {
    subnet 192.168.20.0 netmask 255.255.255.0 {
        option routers 192.168.20.1;
    }
    subnet 192.168.21.0 netmask 255.255.255.0 {
        option routers 192.168.21.1;
    }
}
```

```

    }
    pool {
        range 192.168.20.50 192.168.20.250;
    }
    pool {
        range 192.168.21.50 192.168.21.250;
    }
}

```

9.4. Pools: permitir o denegar clientes

Con **pool** se puede decidir **quién** recibe IP de cada rango. Un ejemplo útil: solo dar IP a los equipos que tienen reserva (**host**) y a nadie más:

```

subnet 192.168.29.0 netmask 255.255.255.0 {
    option routers 192.168.29.1;
    pool {
        deny unknown-clients;      # solo equipos declarados con "host"
        range 192.168.29.100 192.168.29.199;
    }
}

```

ACTIVIDAD 9: configura el pool anterior y comprueba que un cliente sin reserva no recibe IP. Mira en el registro del servidor qué mensaje aparece.

Anexo A. El mismo servicio con Kea

Kea usa un fichero de configuración en **JSON**. Instalación en Debian 12:

```

sudo apt install kea-dhcp4-server
sudo systemctl status kea-dhcp4-server

```

Equivalente de nuestra configuración en `/etc/kea/kea-dhcp4.conf`:

```

{
  "Dhcp4": {
    "interfaces-config": {
      "interfaces": [ "ens19" ],
      "dhcp-socket-type": "raw"
    },
    "authoritative": true,
    "valid-lifetime": 600,
    "max-valid-lifetime": 7200,

    "lease-database": {
      "type": "memfile",
      "persist": true,
      "name": "/var/lib/kea/kea-leases4.csv"
    },

    "option-data": [
      { "name": "domain-name",          "data": "sri.local" },

```

```

    { "name": "domain-name-servers", "data": "1.1.1.1, 8.8.8.8" }
  ],
  "subnet4": [
    {
      "id": 1,
      "subnet": "192.168.29.0/24",
      "pools": [ { "pool": "192.168.29.100 - 192.168.29.199" } ],
      "option-data": [
        { "name": "routers", "data": "192.168.29.1" }
      ],
      "reservations": [
        { "hw-address": "bc:24:11:4c:7f:1d", "ip-address": "192.168.29.50",
"hostname": "pc-04" }
      ]
    }
  ]
}
}
}

```

ISC DHCP (dhcpd.conf)	Kea (kea-dhcp4.conf)
INTERFACESv4 en /etc/default/...	interfaces-config dentro del propio fichero
default-lease-time / max-lease-time	valid-lifetime / max-valid-lifetime
subnet ... netmask ...	subnet4 con "subnet": "red/prefijo"
range	pools
host { hardware ethernet ...; fixed-address ...; }	reservations
dhcpd -t -cf fichero	kea-dhcp4 -t fichero
/var/lib/dhcp/dhcpd.leases	/var/lib/kea/kea-leases4.csv

Validar y aplicar cambios:

```

sudo kea-dhcp4 -t /etc/kea/kea-dhcp4.conf
sudo systemctl restart kea-dhcp4-server
sudo journalctl -u kea-dhcp4-server -f

```

ACTIVIDAD 10: detén `isc-dhcp-server` (`sudo systemctl disable --now isc-dhcp-server`), instala Kea y configúralo con el fichero anterior. Comprueba que los clientes siguen recibiendo la misma configuración y compara el fichero de concesiones de Kea con el de ISC DHCP. ¿Qué formato te parece más fácil de procesar con un script?

Anexo B. Resumen de ficheros y órdenes

Ficheros importantes

Fichero	Para qué sirve	Apartado
<code>/etc/network/interfaces</code>	Configuración de red permanente (<code>ifupdown</code>)	1.3
<code>/etc/resolv.conf</code>	Servidores DNS que usa el equipo	1.4
<code>/etc/hostname</code> y <code>/etc/hosts</code>	Nombre del equipo	1.5
<code>/etc/ufw/sysctl.conf</code>	Activar el reenvío IPv4 (<code>net/ipv4/ip_forward=1</code>)	1.6
<code>/etc/ufw/user.rules</code>	Reglas creadas con la orden <code>ufw</code> (las guarda UFW)	1.7
<code>/etc/ufw/before.rules</code>	Regla de NAT (<code>MASQUERADE</code>) de UFW	1.6
<code>/etc/default/isc-dhcp-server</code>	Interfaz en la que escucha el servidor DHCP	4.1
<code>/etc/dhcp/dhcpd.conf</code>	Configuración del servidor DHCP	5
<code>/var/lib/dhcp/dhcpd.leases</code>	Concesiones asignadas	8
<code>/etc/kea/kea-dhcp4.conf</code>	Configuración de Kea	Anexo A

Órdenes más usadas

Orden	Para qué sirve
<code>ip -br address / ip route</code>	Ver las IP de cada interfaz y la puerta de enlace
<code>sudo ifdown ens19 && sudo ifup ens19</code>	Aplicar cambios de <code>/etc/network/interfaces</code>
<code>sudo ufw status numbered / sudo ufw reload</code>	Ver las reglas de UFW / aplicar cambios en sus ficheros
<code>sudo ufw route allow in on ens19 out on ens18 from 192.168.29.0/24</code>	Permitir que los clientes salgan a través del servidor
<code>sudo ufw allow in on ens19 to any port 67 proto udp</code>	Abrir el puerto del servidor DHCP en la red interna
<code>sudo iptables -t nat -L POSTROUTING -n -v</code>	Ver la regla de NAT y sus contadores

Orden	Para qué sirve
<code>sudo dhcpd -t -cf /etc/dhcp/dhcpd.conf</code>	Comprobar la sintaxis de <code>dhcpd.conf</code>
<code>sudo systemctl restart isc-dhcp-server</code>	Aplicar cambios en el servidor DHCP
<code>sudo journalctl -u isc-dhcp-server -f</code>	Ver en directo el registro del servidor DHCP
<code>sudo dhcp-lease-list</code>	Ver las concesiones en forma de tabla
<code>sudo dhclient -r -v ens18 / sudo dhclient -v ens18</code>	Liberar y pedir IP desde un cliente Linux
<code>ipconfig /release / ipconfig /renew / ipconfig /all</code>	Lo mismo en un cliente Windows
<code>sudo tcpdump -ni ens19 -v port 67 or port 68</code>	Ver el tráfico DHCP